

Robot Operating System

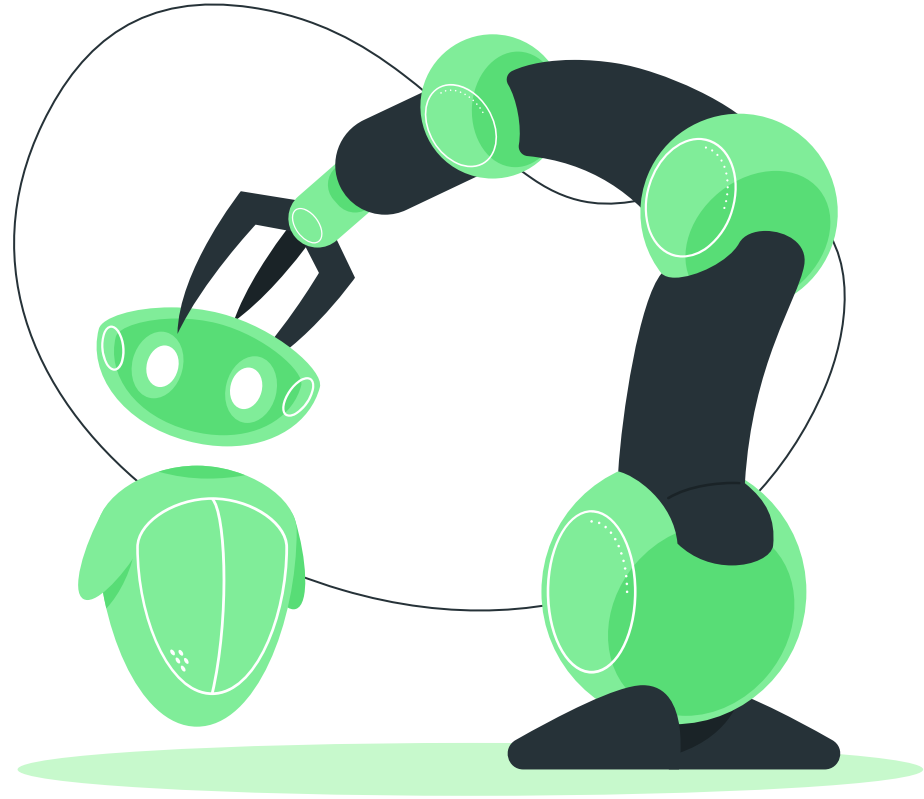


Table of Contents

1

What is ROS?

3

TurtleSim, RQT

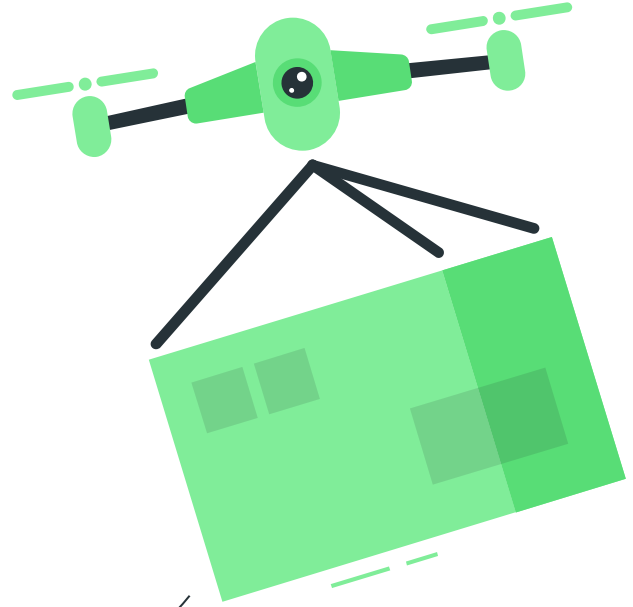
2

**Node, Topics,
Service, Action**

4

Challenges

What is ROS?

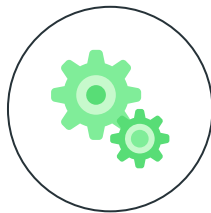


Node, Topics, Service, Action



Node

Piece of code responsible for executing a specific task



Topic

Communication medium between nodes



Service

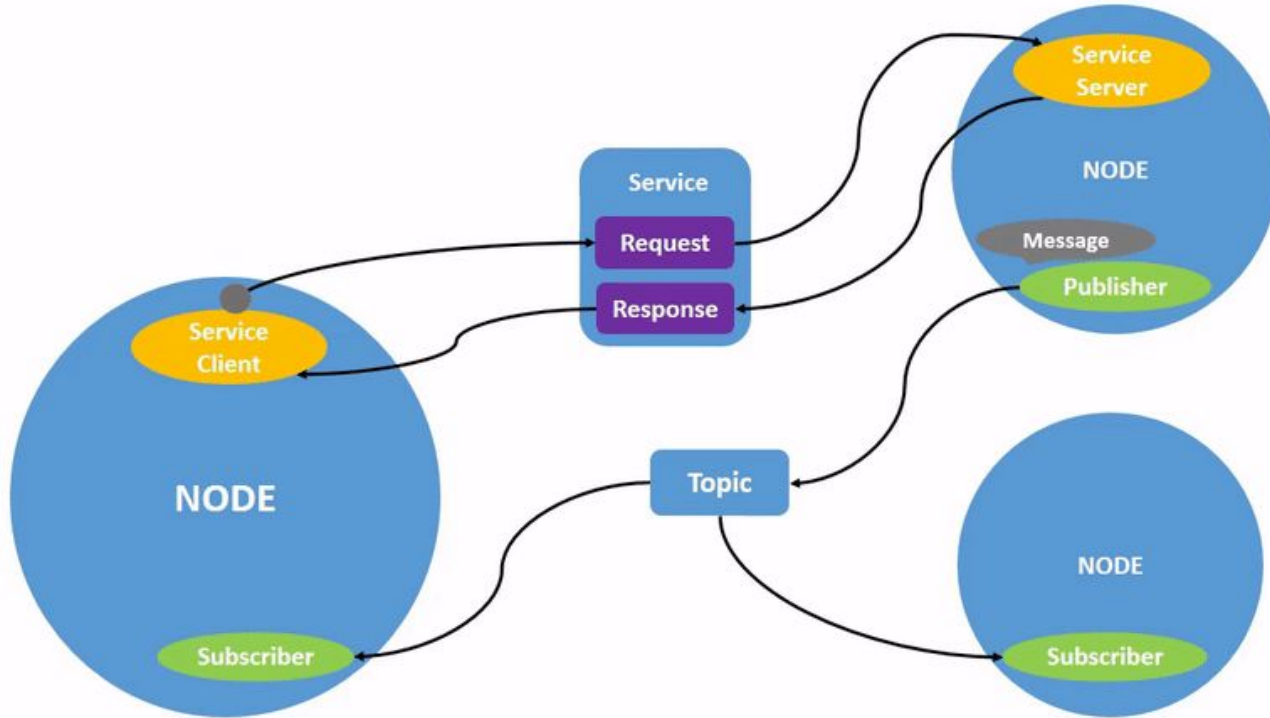
Pontual Communication between nodes



Message

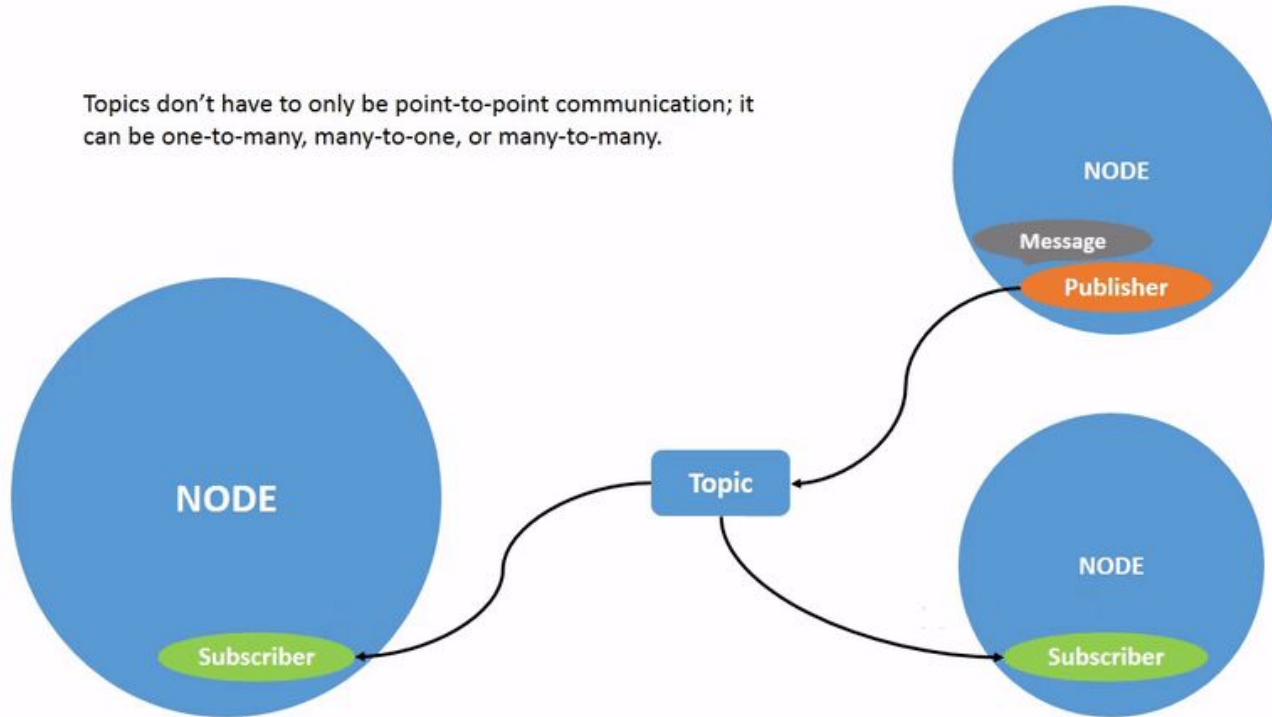
Data structure to communicate between nodes

Node & Package



Topic

Topics don't have to only be point-to-point communication; it can be one-to-many, many-to-one, or many-to-many.



Messages

sensor_msgs/PointCloud2 Message

File: `sensor_msgs/PointCloud2.msg`

Raw Message Definition

```
# This message holds a collection of N-dimensional points, which may
# contain additional information such as normals, intensity, etc. The
# point data is stored as a binary blob, its layout described by the
# contents of the "fields" array.

# The point cloud data may be organized 2d (image-like) or 1d
# (unordered). Point clouds organized as 2d images may be produced by
# camera depth sensors such as stereo or time-of-flight.

# Time of sensor data acquisition, and the coordinate frame ID (for 3d
# points).
Header header

# 2D structure of the point cloud. If the cloud is unordered, height is
# 1 and width is the length of the point cloud.
uint32 height
uint32 width

# Describes the channels and their layout in the binary data blob.
PointField[] fields

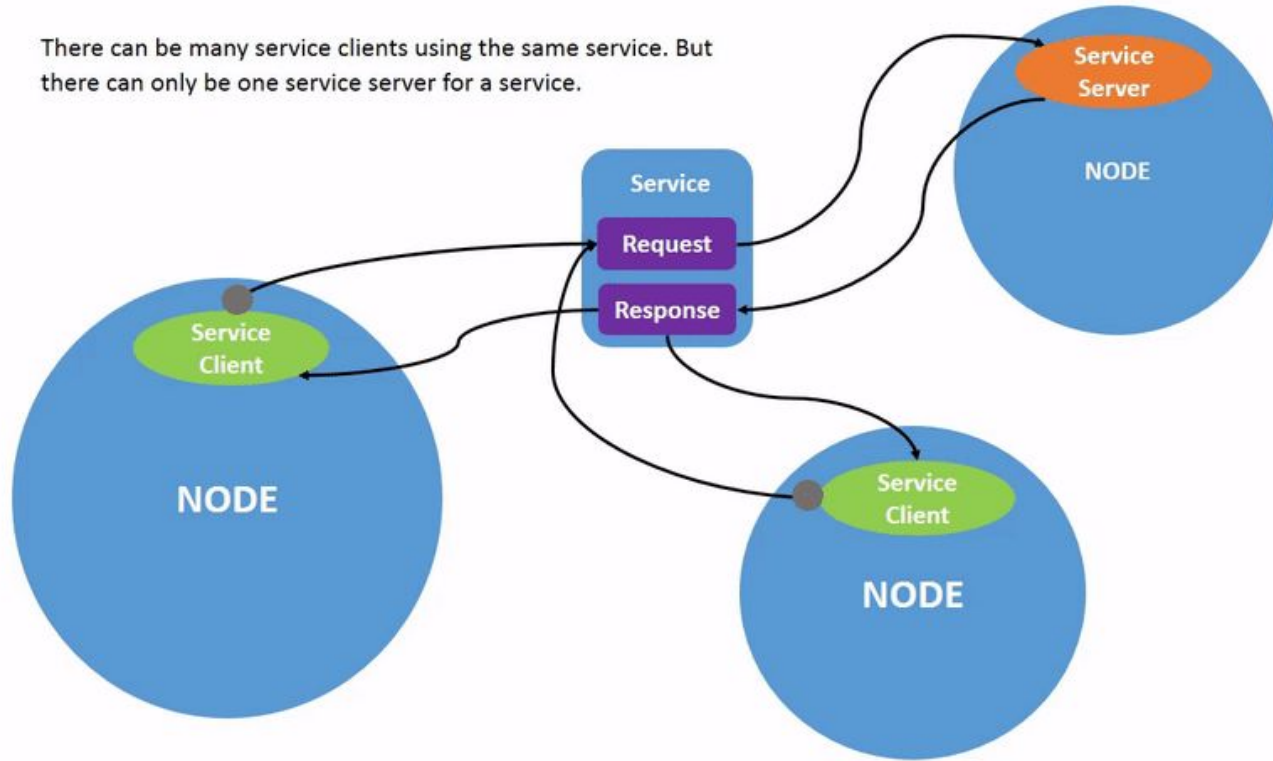
bool    is_bigendian # Is this data bigendian?
uint32  point_step   # Length of a point in bytes
uint32  row_step      # Length of a row in bytes
uint8[] data         # Actual point data, size is (row_step*height)

bool is_dense        # True if there are no invalid points
```

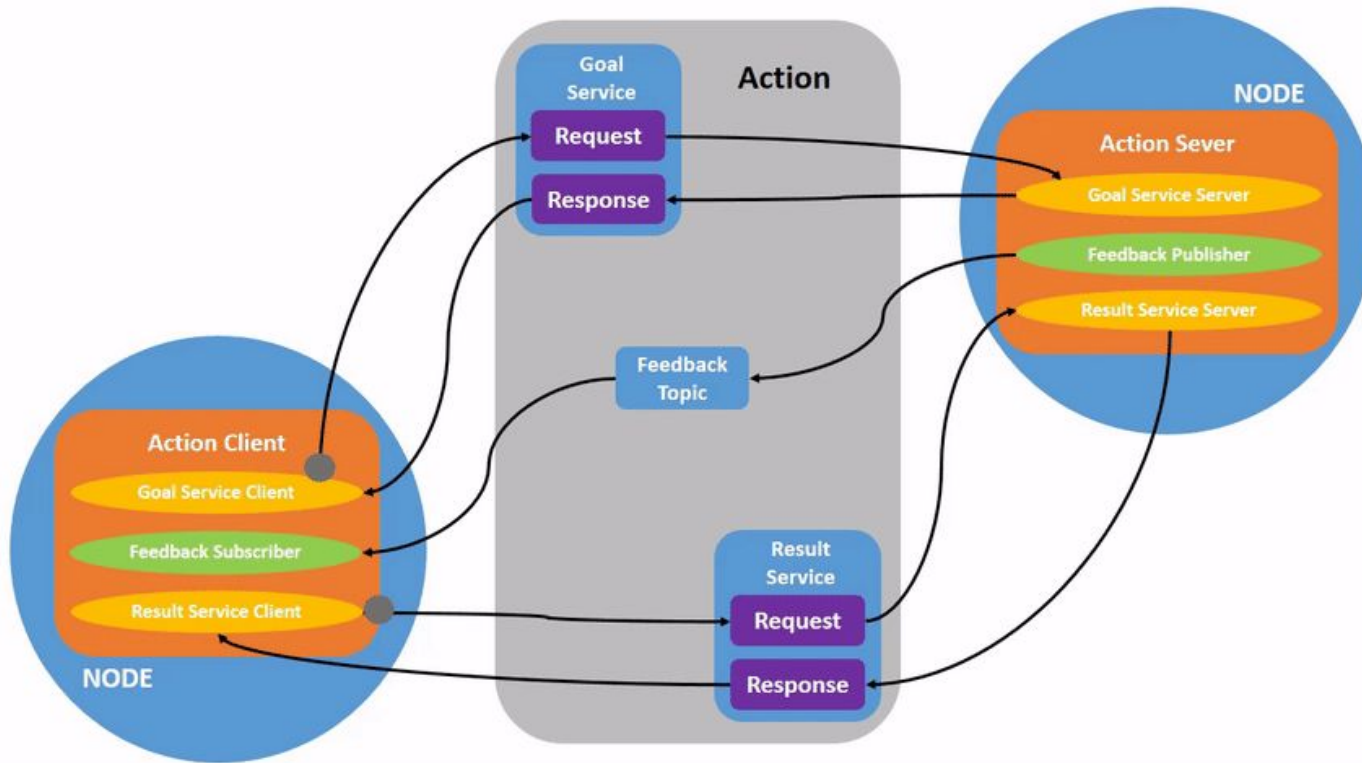
Primitive Type	Serialization	C++	Python2	Python3
bool (1)	unsigned 8-bit int	uint8_t (2)		bool
int8	signed 8-bit int	int8_t		int
uint8	unsigned 8-bit int	uint8_t		int (3)
int16	signed 16-bit int	int16_t		int
uint16	unsigned 16-bit int	uint16_t		int
int32	signed 32-bit int	int32_t		int
uint32	unsigned 32-bit int	uint32_t		int
int64	signed 64-bit int	int64_t	long	int
uint64	unsigned 64-bit int	uint64_t	long	int
float32	32-bit IEEE float	float		float
float64	64-bit IEEE float	double		float
string	ascii string (4)	std::string	str	bytes
time	secs/nsecs unsigned 32-bit ints	ros::Time		rospy.Time
duration	secs/nsecs signed 32-bit ints	ros::Duration		rospy.Duration

Service

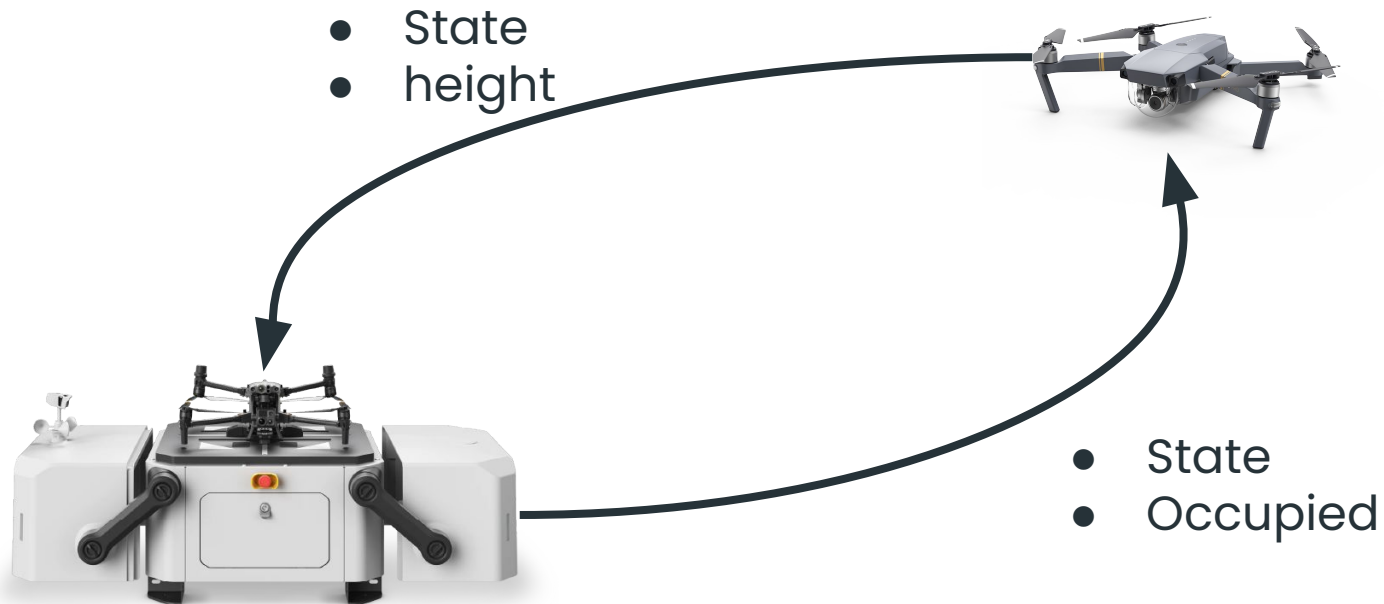
There can be many service clients using the same service. But there can only be one service server for a service.



Action



Real Scenario Application



Commands

Source ROS2:

```
$ source /opt/ros/humble/setup.bash
```

Run a node:

```
$ ros2 run <package_name> <executable_name>
```

Check node list:

```
$ ros2 node list
```

Check node info:

```
$ ros2 node info <node_name>
```

Commands

Check topic list:

```
$ ros2 topic list
```

Subscribe topic:

```
$ ros2 topic echo <topic_name>
```

Publish on topic:

```
$ ros2 topic pub <topic_name> <msg_type> '<args>'
```

Check node info:

```
$ ros2 topic info <topic_name>
```

Commands

Check service list:

```
$ ros2 service list
```

Call service:

```
$ ros2 service call <service_name> <service_type> <arguments>
```

Service type info:

```
$ ros2 service type <service_name>
```

Turtlesim

To use turtlesim:

- Open two terminals;
- Source ROS2 in both of them using:
\$ source /opt/ros/humble/setup.bash
- Run the turtlesim node in one terminal:
\$ ros2 run turtlesim turtlesim_node
- And run the teleop node in the other terminal:
\$ ros2 run turtlesim turtle_teleop_key
- Now play with the previous commands to see the nodes, topics and services running!



**How do I
make a node?**

Install Colcon and create workspace

```
$ sudo apt install python3-colcon-common-extensions
```

Create workspace:

```
$ mkdir -p ~/workspace/src
```

Build ROS2 workspace:

```
$ cd workspace  
$ colcon build
```

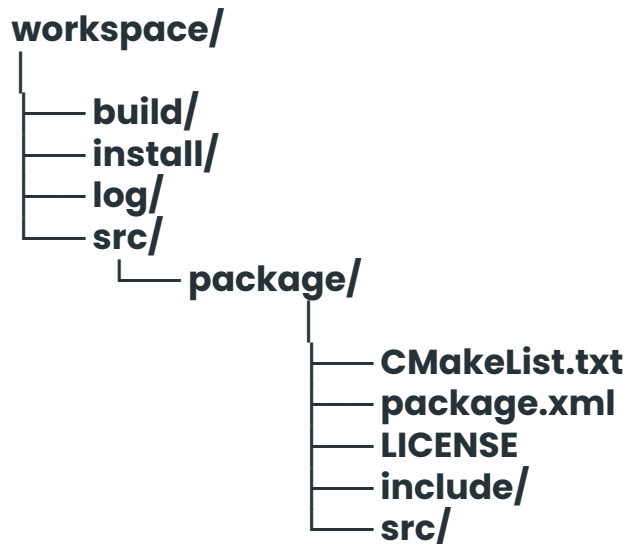
Source ROS2 workspace:

```
$ source install/setup.bash
```

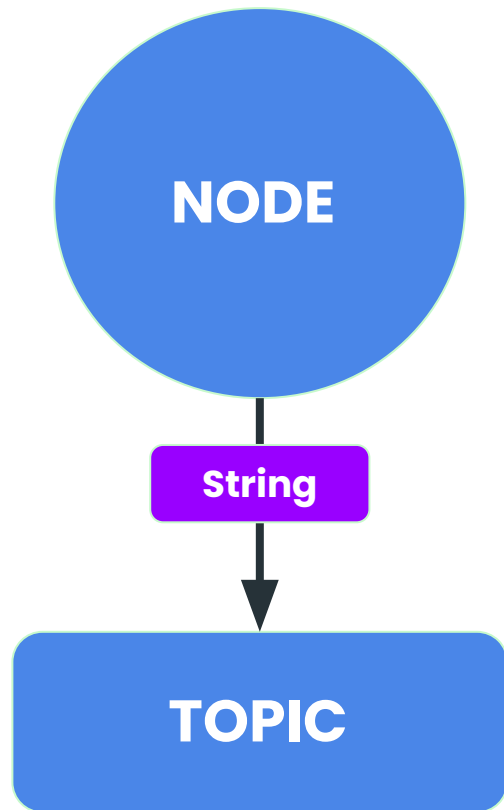

Create Package

Create package:

```
$ cd src  
$ ros2 pkg create --build-type ament_cmake --license Apache-2.0 package
```



Publisher



std_msgs/msg/String Message

File: `std_msgs/msg/String.msg`

Raw Message Definition

```
# This was originally provided as an example message.  
# It is deprecated as of Foxy  
# It is recommended to create your own semantically meaningful message.  
# However if you would like to continue using this please use the equivalent in example_msgs.  
  
string data
```

Compact Message Definition

```
string data
```

Publisher / C++

```
1  #include <iostream>
2  #include <string>
3
4  #include "rclcpp/rclcpp.hpp"
5  #include "std_msgs/msg/string.hpp"
6
7
8  int main(int argc, char* argv[])
9  {
10     rclcpp::init(argc, argv);
11
12     rclcpp::Rate rate(100);
13
14     auto dataStr = std_msgs::msg::String();
15
16     auto node = rclcpp::Node::make_shared("data_publisher");
17
18     auto publisher = node->create_publisher<std_msgs::msg::String>("data/publisher", 1);
19
20     dataStr.data = "data";
21
22     while(rclcpp::ok()){
23
24         publisher->publish(dataStr);
25
26         rate.sleep();
27     }
28
29     rclcpp::shutdown();
30     return 0;
31 }
```

Include ROS2 and the message
you want to work with

Initializes and setup up ROS2
client

Create a message variable

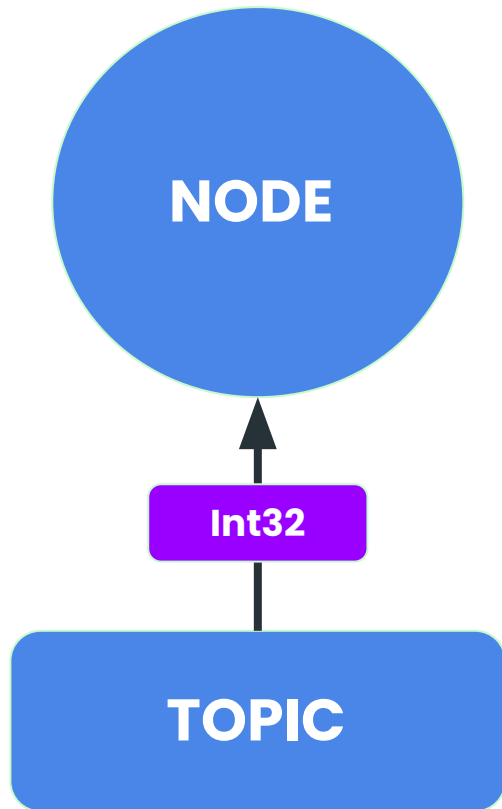
Create the node variable

Create the publisher variable

Publish message on topic

Shutdown ROS2

Subscriber



std_msgs/msg/Int32 Message

File: `std_msgs/msg/Int32.msg`

Raw Message Definition

```
# This was originally provided as an example message.  
# It is deprecated as of Foxy  
# It is recommended to create your own semantically meaningful message.  
# However if you would like to continue using this please use the equivalent in example_msgs.  
  
int32 data
```

Compact Message Definition

```
int32 data
```

Subscriber / C++

```
1  #include <iostream>
2
3  #include "rclcpp/rclcpp.hpp"
4  #include "std_msgs/msg/int32.hpp"
5
6  void dataCallback(const std_msgs::msg::Int32::SharedPtr msg){
7      std::cout << "Data: " << msg->data << std::endl;
8  }
9
10 int main(int argc, char* argv[])
11 {
12     rclcpp::init(argc, argv);
13
14     rclcpp::Rate rate(100);
15
16     auto node = rclcpp::Node::make_shared("data_subscriber");
17
18     auto subscriber = node->create_subscription<std_msgs::msg::Int32>("data/subscriber", 1, dataCallback);
19
20     while(rclcpp::ok()){
21         rclcpp::spin_some(node);
22         rate.sleep();
23     }
24
25     rclcpp::shutdown();
26     return 0;
27 }
28
```

Include ROS2 and the message
you want to work with

Creates the callback
function

Initializes and setup up ROS2
client

Create the node variable

Create the subscriber variable

Runs the callback function

Shutdown ROS2

package.xml

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema" />
3 <package format="3">
4   <name>ros2workshop</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="tiago@todo.todo">tiago</maintainer>
8   <license>Apache-2.0</license>
9
10  <buildtool_depend>ament_cmake</buildtool_depend>
11
12  <depend>roscpp</depend>
13  <depend>std_msgs</depend>
14
15  <test_depend>ament_lint_auto</test_depend>
16  <test_depend>ament_lint_common</test_depend>
17
18  <export>
19    <build_type>ament_cmake</build_type>
20  </export>
21 </package>
```

CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.8)
2 project(package)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 # uncomment the following section in order to fill in
11 # further dependencies manually.
12 # find_package(<dependency> REQUIRED)
13 find_package(rclcpp REQUIRED)
14 find_package(std_msgs REQUIRED)
15
16 add_executable(data_publisher src/dataPublisher.cpp)
17 ament_target_dependencies(data_publisher rclcpp std_msgs)
18
19 add_executable(data_subscriber src/dataSubscriber.cpp)
20 ament_target_dependencies(data_subscriber rclcpp std_msgs)
21
22 install(TARGETS
23   data_publisher
24   data_subscriber
25   DESTINATION lib/${PROJECT_NAME})
26
27
28 if(BUILD_TESTING)
29   find_package(ament_lint_auto REQUIRED)
30   # the following line skips the linter which checks for copyrights
31   # comment the line when a copyright and license is added to all source files
32   set(ament_cmake_copyright_FOUND TRUE)
33   # the following line skips cpplint (only works in a git repo)
34   # comment the line when this package is in a git repo and when
35   # a copyright and license is added to all source files
36   set(ament_cmake_cpplint_FOUND TRUE)
37   ament_lint_auto_find_test_dependencies()
38 endif()
39
40 ament_package()
```

find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

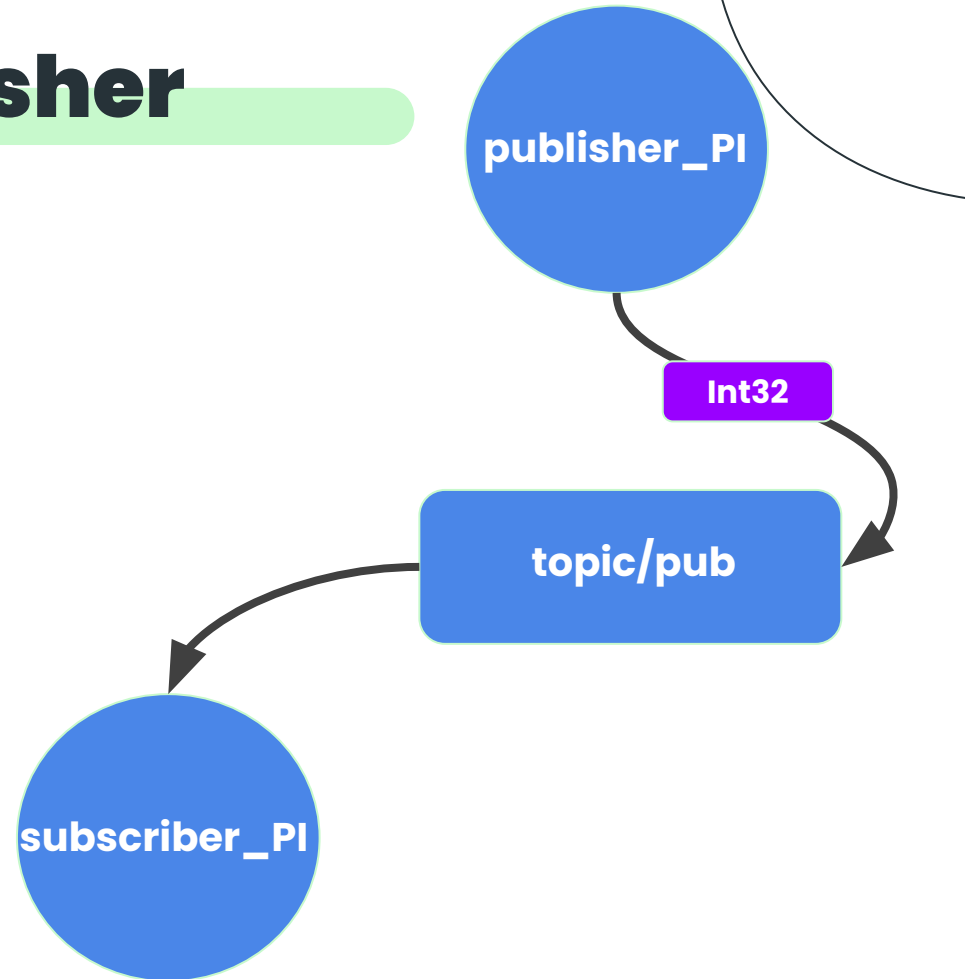
add_executable(data_publisher src/dataPublisher.cpp)
ament_target_dependencies(data_publisher rclcpp
std_msgs)

add_executable(data_subscriber
src/dataSubscriber.cpp)
ament_target_dependencies(data_subscriber rclcpp
std_msgs)



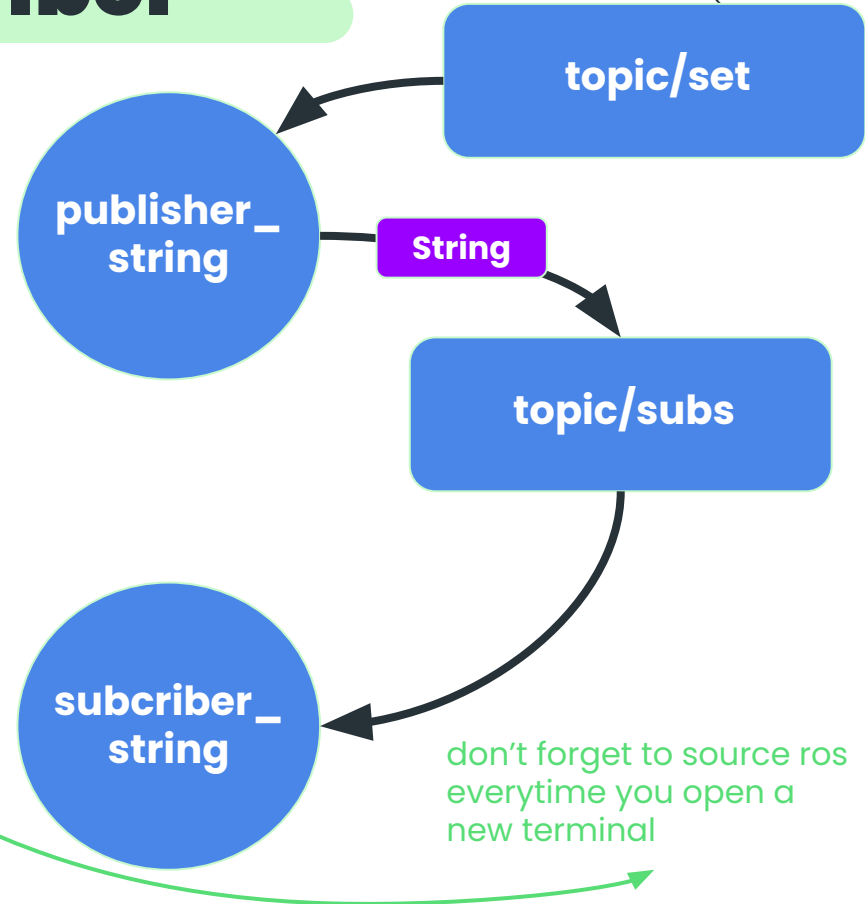
1. Activity – Publisher

1. Complete the code in the node **"publisher_PI"** that publishes the **Int32: "31416"** in the topic **"topic/pub"**;
2. Add the executables in the **CMakeList.txt** file;
3. Compile the workspace using:
\$ colcon build
in the workspace directory;
4. Now source the workspace in two different terminals using:
\$ source install/setup.bash
5. Finally run the publisher and the subscriber using these commands in two different terminals:
\$ ros2 run package sub_pi
\$ ros2 run package pub_pi
6. Check if the subscriber node is printing the correct value!



2. Activity – Subscriber

1. Complete the code of the **"subscriber_string"** node, and make it print the **String data** he is receiving from the **"topic/subs"** topic;
2. Add the executable to the **CMakeList.txt**;
3. Build the workspace in the workspace directory using:
\$ colcon build
4. Source the workspace in two different terminals using:
\$ source install/setup.bash
5. Run the nodes using these commands in the two terminals:
\$ ros2 run package pub_string
\$ ros2 run package sub_string
6. Now the node you created should be printing "You subscribed correctly", try to run this command in a new terminal and see what happens:
\$ ros2 topic pub /topic/set std_msgs/msg/String "data: new_string"



3. Activity – Pub/Subs

1. Write a node that subscribes the Int32 topic "sensor_topic";
2. Convert the message to a String;
3. Publish the string you created in the topic "printer_topic".
4. To see if everything is working just run your node and "printer_node" and "sensor_node".

